Long-Distance Drive-by-Wire Vehicle Control

Remote Vehicle Operation Using a Driving Simulator and ESP32 Communication

ECE 4773: Capstone

Teleoperation Team

Van Lang, Ousmane Segba Kone,

Lili Nguyen, Pundit Vorakitolan

Spring 2026

The UNIVERSITY of OKLAHOMA

Table of Contents

Problem Statement

- 2025 Nissan Leaf EV control is limited by short-range Bluetooth
- This project enables long-distance Drive-by-Wire control
- Inputs will control steering, acceleration, and braking remotely

System Overview

- Controller or simulator sends input
- Software processes the input
- ESP32 sends the commands
- Long-distance link carries the commands
- ESP32 receives the commands
- Vehicle follows the commands

Hardware

- Logitech G29 simulator
- Sender laptop
- ESP32 transmitter and receiver
- LTE or Wi-Fi communication link
- 2025 Nissan Leaf with Drive-by-Wire system
- Cameras

Software

- Sender
  - Python script using Pygame to receive input at 60Hz frequency
  - Translates steering axis from –1.0 to 1.0 and 0 to 1.0 for the pedals
  - UDP protocol to prioritize speed over retransmission

- Server
  - OCI server hosted in Chicago to coordinate remote connections
  - Using "heartbeat" packets to bypass firewall
  - ESP32 receives G29 data here

- Arduino
  - ESP32 connects to Wi-Fi and receives UDP driving commands for steering, throttle, and brake.
  - Reads steering feedback from the CAN bus and uses a PID controller to match the target steering angle.
  - Outputs calibrated DAC voltages to control steering torque, accelerator, and brake signals.

Integration Strategy

- Frequency of 60Hz (16.6ms)to match hardware requirements
- Sequence numbering to handle UDP packet reordering and eliminate old data
- Unidirectional command stream from G29 to OCI with bidirectional heartbeat persistence
- Synchronizing cloud packets with local CAN-Bus telemetry for PID stability
- Optimized "hole-punching" to ensure real-time command delivery across the differing networks

Demonstration

- In lab: Operator sends steering, throttle, and brake commands from the G29 control system.
- In car: ESP32 receives commands, reads CAN feedback, and outputs control signals to the car.
- Roles: One person operates the lab controls; one person monitors the car and safety.

Results

- Accuracy: Vehicle follows control inputs correctly and no mismatch.
- Latency: Fast response between lab command and car action. Roughly < 180 ms
- Reliability: Stable connection maintained throughout the demo and no drops when testing at long distance

Challenges & Solutions

- Inverse Control Logic:
  - Resolved a mapping error where gas and brake inputs were swapped
- Directional Asymmetry:
  - Issues turning right
- Distributed System Management:
  - Synchronizing three environments: Python (Sender), Python (Cloud Relay), and C++ (ESP32)
- Control Sensitivity (Torque):
  - Moving the G29 wheel slightly, but not mapping correctly in the Nissan Leaf

Future Work

- Improve latency
- Blinkers
- Live video monitor hardware/accessible GUI
- Film cool car video with Dr. Xu

Conclusion

- Hosted cloud relay for long-distance communication
- Sent G29 driving inputs to the vehicle-side system
- Controlled steering, acceleration, and braking remotely
- Implemented heartbeat and failsafe safety logic
- Built a working foundation for future teleoperation testing
- Built a working foundation for future teleoperation testing